

Applied Deep Learning

Chapter 1: Preliminaries

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Machine Learning (ML)

Goodfellow et al. *informally* define ML as “. . . a form of applied statistics with increased emphasis on the use of computers to statistically estimate complicated functions. . .”

Though good, this definition is still unclear! Let's put it in simple words

For most problems, there are two approaches to get to a solution

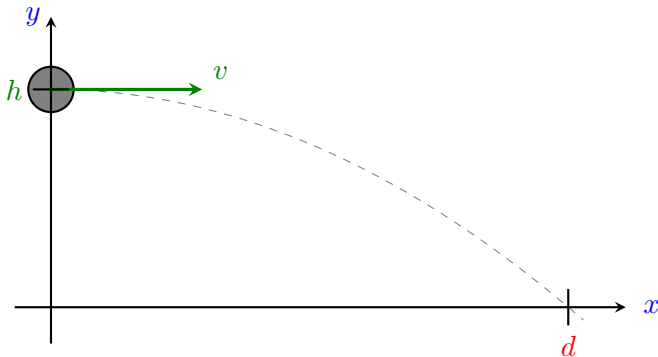
- through **analytic** derivation
- using **data-driven** algorithms

ML develops efficient **data-driven** algorithms for *complicated problems* whose analysis is either **infeasible** or **too complicated**

Let's try a **dummy example**!

Motion of a Projectile

A projectile, located at height h , is initiated by horizontal velocity v . We are asked to determine horizontal distance d at which it hits the ground



Motion of a Projectile: *Analytic Solution*

You may remember the analytic solution from high school or general physics

At time t , the location of the projectile (x, y) is

$$x = vt \qquad y = -\frac{g}{2}t^2 + h$$

with $g \approx 9.8 \text{ m/s}^2$ being the gravitational acceleration. When it hits the ground, we have $y = 0$; thus, we can find the hitting time as

$$t_0 = \sqrt{\frac{2h}{g}} = \gamma\sqrt{h}$$

At $t = t_0$, we have $x = d = vt_0$. Defining $\gamma = \sqrt{2/g} \approx 0.45$, we have

$$d = \gamma v\sqrt{h}$$

Motion of a Projectile: *Analytic Solution*

The projectile hits the ground at the horizontal distance

$$d = \gamma v \sqrt{h}$$

To derive this analytic result, we have used two facts

- We knew Newton's laws that describe the motion
- We could solve the equations for d analytically

Well! This is not the case in all problems! In fact, there might be

- **no analytic law known** that describes the relations, or
- a **complicated law** whose analysis is computationally **infeasible**

Let's now look at the **ML** approach!

Motion of a Projectile: *ML Approach*

A computer scientist could well have forgotten general physics

We conduct the experiment for I different times: in try $i = 1, \dots, I$

- we initiate with different **velocity** v_i and **height** h_i
- we measure the **horizontal distance** d_i

We then **assume** that d_i and $[v_i, h_i]$ are **related** via a *pre-defined function*; for instance, we assume d_i and $[v_i, h_i]$ are related as

$$d_i = w_0 v_i + w_1 h_i$$

We then try to find the **optimal** values

$$w_0 = w_0^* \text{ and } w_1 = w_1^*$$

such that *pre-defined function closely matches our experimental results*

Motion of a Projectile: *ML Approach*

How can we find $w_0^\star$ and $w_1^\star$?

The function should match our experimental data, i.e., *for any i*

$$d_i \stackrel{!}{=} w_0^\star v_i + w_1^\star h_i \rightsquigarrow (d_i - w_0^\star v_i - w_1^\star h_i)^2 \stackrel{!}{=} 0$$

By $\stackrel{!}{=}$, we mean that we intend to have this identity holding

Having this identity *for any i* is equivalent to write

$$\sum_{i=1}^I (d_i - w_0^\star v_i - w_1^\star h_i)^2 \stackrel{!}{=} 0$$

But, such $w_0^\star$ and $w_1^\star$ do not necessarily exist if $I \geq 3$!

Don't worry if you don't see it right away! You'll see it in an assignment!

Motion of a Projectile: *ML Approach*

How can we find w_0^* and w_1^* ? We find w_0^* and w_1^* such that

$$\sum_{i=1}^I (d_i - w_0^* v_i - w_1^* h_i)^2$$

is as small as possible

Let us define the *loss* $\mathcal{L}(w_0, w_1)$ as

$$\mathcal{L}(w_0, w_1) = \sum_{i=1}^I (d_i - w_0 v_i - w_1 h_i)^2$$

We then find w_0^* and w_1^* that minimize the *loss*

$$(w_0^*, w_1^*) = \underset{w_0, w_1}{\operatorname{argmin}} \mathcal{L}(w_0, w_1)$$

Motion of a Projectile: *ML Approach*

For new given v and h , we find the **horizontal distance** as

$$d = w_0^* v + w_1^* h$$

We could intuitively say that

- The ML-derived distance is **not as accurate** as the analytic one
 - ↳ The assumed relation between d and $[v, h]$ is not exact
- The ML approach gets **better** as we **increase the number of trial I**

In the first assignment, we will program this dummy example!

Motion of a Projectile: *ML Approach*

The ML approach has *three main components*:

- 1 *Dataset*: For our example, we collected *dataset* $\mathbb{D}$

$$\mathbb{D} = \{([v_i, h_i], d_i) : i = 1, \dots, I\}$$

- 2 *Model*: We assumed d and $[v, h]$ are related via a *linear model*

$$d = w_0 v + w_1 h$$

- 3 *Loss*: We evaluated the loss of our model for given w_0 and w_1 as

$$\mathcal{L}(w_0, w_1) = \sum_{i=1}^I (d_i - w_0 v_i - w_1 h_i)^2$$

We now take a deeper look into each component

Recap: Vectors and Matrices

We're going to use frequently linear algebra! So, let's recall some basics

$\mathbf{x} \in \mathbb{R}^N$ is an N -dimensional *column*-vector with N *real* entries, i.e.,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \rightsquigarrow \mathbf{x}^\top = [x_1, \dots, x_N]$$

If we want to make a *row*-vector, we *transpose* it, i.e., use $\mathbf{x}^\top$

Notation

We show vectors with **bold-face** small letters and drop *column/row*

- A vector is **by default** a *column-vector*
- If we need a *row*-vector, we *transpose* its *column* version

Recap: Vectors and Matrices

Matrix $\mathbf{A} \in \mathbb{R}^{N \times M}$ can be seen as

- either as the *collection of M column-vectors of dimension N*

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_M]$$

with $\mathbf{a}_m \in \mathbb{R}^N$ for $m = 1, \dots, M$

- or as the *collection of N row-vectors of dimension M*

$$\mathbf{A} = \begin{bmatrix} \bar{\mathbf{a}}_1^\top \\ \vdots \\ \bar{\mathbf{a}}_N^\top \end{bmatrix}$$

with $\bar{\mathbf{a}}_n \in \mathbb{R}^M$ for $n = 1, \dots, N$

Recap: Vectors and Matrices

Notation

We show matrices with **bold-face** capital letters

Two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^N$ of *same dimension* N are *inner-multiplied* as

$$\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x} = \sum_{n=1}^N x_n y_n$$

They can further *outer-multiplied* as

$$\mathbf{x} \mathbf{y}^\top = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} [y_1, \dots, y_N] = \begin{bmatrix} x_1 y_1 & \dots & x_1 y_N \\ \vdots & \dots & \vdots \\ x_N y_1 & \dots & x_N y_N \end{bmatrix} = \left(\mathbf{y} \mathbf{x}^\top \right)^\top$$

Recap: Vectors and Matrices

Multiplying matrix $\mathbf{A} \in \mathbb{R}^{N \times M}$ with $\mathbf{x} \in \mathbb{R}^M$ can be seen as

- either as the *linear combination of column-vectors in $\mathbf{A}$*

$$\mathbf{Ax} = [\mathbf{a}_1, \dots, \mathbf{a}_M] \begin{bmatrix} x_1 \\ \vdots \\ x_M \end{bmatrix} = \sum_{m=1}^M x_m \mathbf{a}_m \in \mathbb{R}^N$$

- or as the *collection of inner-products of row-vectors with $\mathbf{x}$*

$$\mathbf{Ax} = \begin{bmatrix} \bar{\mathbf{a}}_1^\top \\ \vdots \\ \bar{\mathbf{a}}_N^\top \end{bmatrix} \mathbf{x} = \begin{bmatrix} \bar{\mathbf{a}}_1^\top \mathbf{x} \\ \vdots \\ \bar{\mathbf{a}}_N^\top \mathbf{x} \end{bmatrix} \in \mathbb{R}^N$$

ML Components: *Dataset*

Dataset is the collection of **data-points**; in our example, the dataset was

$$\mathbb{D} = \{([v_i, h_i], d_i) : i = 1, \dots, I\}$$

which has I **data-points** $([v_i, h_i], d_i)$ for $i = 1, \dots, I$

This is an example of a **labeled dataset**: a dataset whose data-points contain both the **inputs** and their corresponding **labels (outputs)**

More general, a **labeled dataset** with I data-points is

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

- $\mathbf{x}_i \in \mathbb{R}^N$ is an input vector with N entries
- y_i is the **label** of $\mathbf{x}_i$ that is a scalar

ML Components: *Supervised Learning*

When the **dataset** is **labeled**, our *task* is clear:

- We observe I inputs to a **function** with their **outputs (labels)**
- We try to find out (**learn**) this **function**

Our task is a **learning task**, because we are trying to

learn the unknown relation between **inputs** and **labels**

It describes a **supervised learning problem**, since

a **supervisor** has filled our dataset with **labels**

In other words, for the sample **inputs** in our hand, we know the **outputs**

*In this course, we are mainly focused on **supervised learning***

ML Components: *Unsupervised Learning*

- + But can a *dataset* be *unlabeled*?
- Yes! This is the case in *unsupervised learning* in which we are to *learn features* of a *data-point* x by investigating a set of its samples

In this case, the dataset is of the form

$$\mathbb{D} = \{x_i : i = 1, \dots, I\}$$

which is *unlabeled*. Our *learning task* is further *unsupervised*

Let's make it crystal clear via an example!

ML Components: *Example of Unsupervised Learning*

An N -dimensional **data-point** $\mathbf{x}$ is coming from a natural process, e.g., it contains the pixel values of an image taken from body. It is known that $\mathbf{x}$ is a linear combination of $Q \ll N$ **principal vectors**: any $\mathbf{x}$ is

$$\mathbf{x} = \sum_{q=1}^Q a_q \mathbf{v}_q = [\mathbf{v}_1, \dots, \mathbf{v}_Q] \begin{bmatrix} a_1 \\ \vdots \\ a_Q \end{bmatrix} = \mathbf{V} \mathbf{a}$$

with $\mathbf{a} \in \mathbb{R}^Q$ and $\mathbf{V} \in \mathbb{R}^{N \times Q}$. Nevertheless, we do **not** know the **principle vectors**, i.e., **matrix $\mathbf{V}$ is unknown to us**.

Our **learning task** is to

find out **what $\mathbf{V}$ is** by investigating **I samples of $\mathbf{x}$**

ML Components: *Example of Unsupervised Learning*

In this problem, our **dataset** is of the form

$$\mathbb{D} = \{\mathbf{x}_i : i = 1, \dots, I\}$$

which is **unlabeled**. We know that these samples are of the form

$$\mathbf{x}_i = \mathbf{V} \mathbf{a}_i$$

but we know *neither* $\mathbf{V}$ *nor* $\mathbf{a}_i$. This is an **unsupervised** learning problem

This is the well-known problem of *dimensionality reduction*

- we get a large dimensional vector $\mathbf{x}$
- we derive a **feature** out of it, i.e., $\mathbf{a}$, which is of *lower* dimension

The classical solution is given by **Principal Component Analysis (PCA)**

ML Components: *Unsupervised* $\rightsquigarrow$ *Supervised*

Supervised and **unsupervised** are not only divisions in terms of **dataset**:

- We may deal with a **semi-supervised learning** task
 - ↳ **Dataset** contains both **labeled** and **unlabeled** data-points
- We may deal with a **reinforcement learning** task
 - ↳ An **agent** is to learn a set of actions each relying on the others
 - ↳ **Agent's dataset** grows **through interactions** with the environment
 - ↳ Best example is the design of a machine that learns to play chess

As mentioned, the main focus of this course is on **supervised** learning

We will also discuss **unsupervised** later in the course

Reinforcement learning is beyond the scope of this course

There is a separate ECE course on this topic next

Stay tuned for that!

ML Components: *Model*

Now, we know what *supervised learning* and *labeled dataset* are

Let's assume we are given by the *labeled dataset*

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

and are to learn the function that relates an *input* $\mathbf{x}$ to its *label* y

For this task we need to consider a *Model*

Model is a *parameterized* function that is used to describe the relation between the *input* $\mathbf{x}$ and its *label* y

In our *dummy example*, the model was *linear*

$$\text{label} = d = w_0 v + w_1 h = [w_0, w_1] \begin{bmatrix} v \\ h \end{bmatrix} = \mathbf{w}^T \mathbf{x}$$

Recap: Linear versus Affine Function

Linear Function

Scalar $y \in \mathbb{R}$ is a linear function of vector $\mathbf{x} \in \mathbb{R}^N$ if

$$y = \mathbf{w}^T \mathbf{x}$$

for some constant vector $\mathbf{w} \in \mathbb{R}^N$

Affine Function

Scalar $y \in \mathbb{R}$ is an affine function of vector $\mathbf{x} \in \mathbb{R}^N$ if

$$y = \mathbf{w}^T \mathbf{x} + b$$

for some constant vector $\mathbf{w} \in \mathbb{R}^N$ and scalar $b \neq 0$

Recap: Linear versus Affine Function

Key difference: If $\mathbf{x} = \mathbf{0}_N$ is the vector of all zeros

- **Linear function** returns **zero**: it passes *through the origin*
- **Affine function** returns **non-zero**: it does *not* pass through the origin

We can simply extend the definition to a **vector-valued** functions

Let $\mathbf{A} \in \mathbb{R}^{M \times N}$ and $\mathbf{b} \in \mathbb{R}^M$; then,

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

is a linear **vector-valued** function of $\mathbf{x}$ and

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$$

is an affine **vector-valued** function

ML Components: *Model*

In our **dummy example**, we considered a **linear** model, but

- we could have considered an affine model

$$\text{label} = b + \mathbf{w}^T \mathbf{x}$$

- or a polynomial model

$$\text{label} = b + \mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^2 + \dots + \mathbf{w}_P^T \mathbf{x}^P$$

Notation

By $f(\mathbf{x})$ we refer to entry-wise function operation

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \rightsquigarrow \mathbf{x}^p = \begin{bmatrix} x_1^p \\ \vdots \\ x_N^p \end{bmatrix} \quad \text{or} \quad \sqrt{\mathbf{x}} = \begin{bmatrix} \sqrt{x_1} \\ \vdots \\ \sqrt{x_N} \end{bmatrix}$$

ML Components: *Model*

In our **dummy example**, we considered a **linear** model, but

- we could have considered even a wired model

$$\text{label} = \left(\mathbf{w}_1^T \mathbf{x} \right) \left(\mathbf{w}_2^T \sqrt{\mathbf{x}} \right)$$

*Models are always **parameterized** meaning that*

*they contain some **parameters** that are to be **tuned***

Example: $\mathbf{w}$ in linear model or $\mathbf{w}_p$'s and b in polynomial model

Model parameters are of two types:

- **Hyperparameters**
- **Learnable parameters**

ML Components: *Model*

Hyperparameters

Parameters that are required to specify the model *explicitly*

Best example is the **order** P in the polynomial model: we need to know P in order to write down the model explicitly

If we know $P = 2$; then, we know that our model is

$$\text{label} = b + \mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^2$$

Hyperparameters are specified prior to the start of learning process

- + How can we **tune** them?
- We will discuss it in detail in this course! For the moment, assume that *they are given to us by some genie*

ML Components: *Model*

Learnable Parameters

Once the **hyperparameters** are set, we have a model with some **parameters** that are to be **learned**, such that the model fits our dataset

In our polynomial example after we set $P = 2$, we get the model

$$\text{label} = b + \mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^2$$

Now we need to **learn** $\mathbf{w}_1$, $\mathbf{w}_2$, and b : *let our dataset be*

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

we need to find (learn) values $\mathbf{w}_1^$, $\mathbf{w}_2^*$, and b^* such that for all i*

$$y_i \approx b^* + \mathbf{w}_1^{*\top} \mathbf{x}_i + \mathbf{w}_2^{*\top} \mathbf{x}_i^2$$

ML Components: *Model*

Learnable Parameters

Once the **hyperparameters** are set, we have a model with some **parameters** that are to be **learned**, such that the model fits our dataset

In our polynomial example with $P = 2$: we learn w_1^ , w_2^* , and b^* such that*

$$y_i \approx b^* + w_1^{*\top} x_i + w_2^{*\top} x_i^2$$

- + But what does this $\approx$ mean?
- This approximation needs to be quantified!

*This is what the **loss function** does for us*

- + How do we **learn** the **learnable parameters**?
- We answer this after we understand the **loss function**

ML Components: Loss

Loss Function

Loss function quantifies the difference between the *output of the model* and the *true label*

Back to our polynomial example with dataset $\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$

- We set the **hyperparameter** P to $P = 2$
- We now set the **learnable parameters** to

$$\mathbf{w}_1 = \mathbf{w}_1^{(0)} \quad \mathbf{w}_2 = \mathbf{w}_2^{(0)} \quad b = b^{(0)}$$

If we give the data-point $\mathbf{x}_i$ to this model as input, it returns

$$z_i = b^{(0)} + \mathbf{w}_1^{(0)\top} \mathbf{x}_i + \mathbf{w}_2^{(0)\top} \mathbf{x}_i^2$$

ML Components: Loss

Back to our polynomial example with dataset $\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$

$$z_i = b^{(0)} + \mathbf{w}_1^{(0)\top} \mathbf{x}_i + \mathbf{w}_2^{(0)\top} \mathbf{x}_i^2$$

Loss determines the difference between z_i and true label y_i in $\mathbb{D}$

$$\mathcal{L}(z_i, y_i) = \ell_i \in \mathbb{R}$$

Let's see few examples of loss function:

We can calculate the *squared error*

$$\mathcal{L}(z_i, y_i) = (z_i - y_i)^2$$

which intuitively determines the *energy of the difference*

ML Components: Loss

Back to our polynomial example with dataset $\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$

$$z_i = b^{(0)} + \mathbf{w}_1^{(0)\top} \mathbf{x}_i + \mathbf{w}_2^{(0)\top} \mathbf{x}_i^2$$

Loss determines the difference between z_i and true label y_i in $\mathbb{D}$

$$\mathcal{L}(z_i, y_i) = \ell_i \in \mathbb{R}$$

Let's see few examples of loss function:

We can calculate the error indicator

$$\mathcal{L}(z_i, y_i) = \mathbb{1}(z_i \neq y_i) = \begin{cases} 1 & z_i \neq y_i \\ 0 & z_i = y_i \end{cases}$$

which indicates the occurrence of error

ML Components: A Quick Wrap-up

Any ML problem has *three components*:

- 1 *Dataset* which is the collection of samples

supervised learning

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

- 2 *Model* that describes the relation between the *input* and *label*

$\mathbf{h}$: Hyperparameters

$$z = f_{\mathbf{h}}(\mathbf{x}|\mathbf{w})$$

$\mathbf{w}$: Learnable Parameters

- 3 *Loss* quantifies the difference between the *model's output* and *true label*

$$\mathcal{L}(z_i, y_i)$$

What's Next?

Now, we know the components of an ML problem!

- + *How can we solve the problem? Or, speaking in the language of ML people, how can we address the learning task?*
- Recall that the learning task is to **tune the learnable parameters** ($\mathbf{w}$ in the last slide): once we tune them, the problem is over

We approximate the label of a new input $\mathbf{x}_{\text{new}}$ as $y_{\text{new}} = f_{\mathbf{h}}(\mathbf{x}_{\text{new}} | \mathbf{w})$

The process of **tuning the learnable parameters** is called
training of the model

- + *How do we do the **training**?*
- We see it very shortly, but first we need a more serious example!

Classification

Classification is a supervised learning problem in which

labels belong to a discrete set: $y_i \in \{c_1, \dots, c_J\}$

the label y_i represents the class to which the input x_i belongs

Best example is image classification:

- The dataset contains some images and their labels

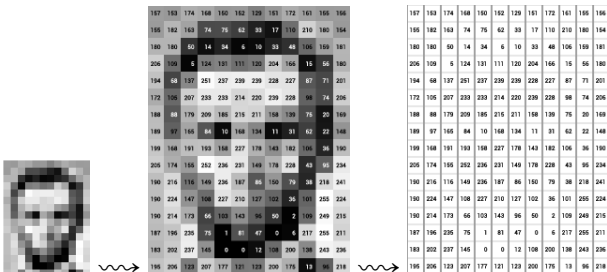
$$\mathbb{D} = \{(\text{image 1, dog}), (\text{image 2, cat}), \dots, (\text{image } I, \text{cat})\}$$

- The images are either *dog* or *cat*

We can simply convert $\mathbb{D}$ into a numerical dataset

Classification: *Image Classification*

Any image is nothing but a *collection of pixel values*



We will work with images a lot in this course, so we will learn the meaning of *pixel values* very well. For now, the important thing is that

*for each image, we can make a vector containing its *pixel values**

Thus, *image i* in $\mathbb{D}$ can be shown by a *pixel vector x_i*

Classification: *Image Classification*

The *labels* can further be marked by integer numbers as *classes*

In our example, we have two classes of *dog* and *cat*: so, we could say

$$\text{label of } \textcolor{red}{\text{dog}} \text{ images} = 0 \quad \text{and} \quad \text{label of } \textcolor{red}{\text{cat}} \text{ images} = 1$$

So, a collection of I images like

$$\mathbb{D} = \{(\text{image } 1, \textcolor{red}{\text{dog}}), (\text{image } 2, \textcolor{red}{\text{cat}}), \dots, (\text{image } I, \textcolor{red}{\text{cat}})\}$$

can be represented by the numerical dataset

$$\mathbb{D} = \{(\textcolor{green}{x}_1, 0), (\textcolor{green}{x}_2, 1), \dots, (\textcolor{green}{x}_I, 1)\}$$



pixel vector of image 1 it's an image of a dog

Classification: *Binary Classification*

We now start with the basic case of binary classification

Binary Classification

A classification problem with only two classes, i.e., $y_i \in \{0, 1\}$

Our example was a binary classification with *dog:0* and *cat:1*

Binary classification is very fundamental, since

- it is one of the *very first problems* investigated in ML
- other classification problems are reduced into a *series of binary classifications*: say we want to classify an image as *dog, cat or car*
 - ▶ *Binary Classification 1*: Is it *class 0: dog* or *class 1: {cat, car}*?
 - ↳ If *class 0* $\rightsquigarrow$ classification ended
 - ↳ If *class 1* $\rightsquigarrow$ *Binary Classification 2*: Is it *class 0: cat* or *class 1: car*?

Let's build the main components of this ML problem

Binary Classification: *Dataset*

Dataset is similar to what we had in our **dog** or **cat** example

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

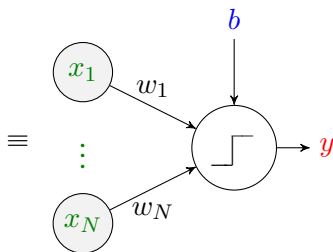
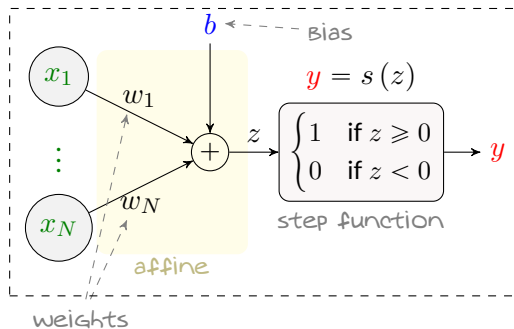
- $\mathbf{x}_i \in \mathbb{R}^N$ is a real-valued N -dimensional vector
 - ▶ For instance it contains the **pixel values** of an image with N pixels
- $y_i \in \{0, 1\}$ is a **binary label**

Binary Classification: *Model*

- + What should our *model* be?
- The *model* gets N real numbers and returns a **binary number**

Classification models are also called *Classifiers*

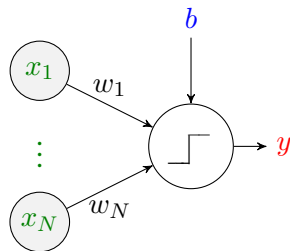
Let's start with *perceptron* who is the mother of *neural networks*



Binary Classification: *Perceptron*

Perceptron is a *linear classifier*

- it determines an *linear transform* of $\mathbf{x}$
- it *classifies* using the sign of this *transform*

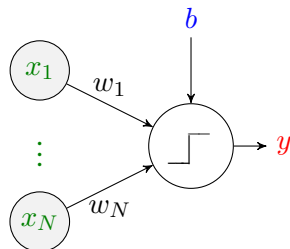


Mathematically, the perceptron is described by

$$y = s\left(\sum_{n=1}^N w_n x_n + b\right) = s\left(\underbrace{[w_1, \dots, w_N]}_{\mathbf{w}^T} \underbrace{\begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}}_{\mathbf{x}} + b\right) = s(\mathbf{w}^T \mathbf{x} + b)$$

Binary Classification: *Perceptron*

$$y = s(\mathbf{w}^T \mathbf{x} + b)$$

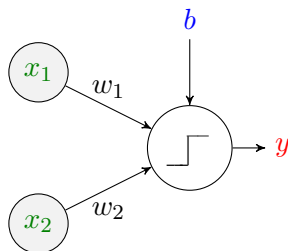


- + Is there any **hyperparameter** in this model?
- We could look at **step function** as a **hyperparameter**: we could have chosen another function to map the **affine transform** to a **binary label**
- + Is there any **learnable parameter** in this model?
- Of course! $\mathbf{w}$ and b are the **learnable parameters**

Perceptron: Geometrical Interpretation

Perceptron geometrically realizes a *linear division of $\mathbb{R}^N$*

To see this, let's focus on the two-dimensional case, i.e., $N = 2$



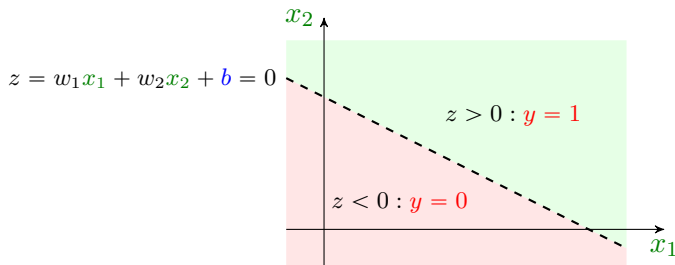
$$y = s(\mathbf{w}^T \mathbf{x} + b) = s(w_1 x_1 + w_2 x_2 + b) = \begin{cases} 1 & \text{if } w_1 x_1 + w_2 x_2 + b \geq 0 \\ 0 & \text{if } w_1 x_1 + w_2 x_2 + b < 0 \end{cases}$$

Perceptron: Geometrical Interpretation

Let's go to the two-dimensional space with one dimension denoting x_1 and the other x_2 and plot the line

$$z = w_1 x_1 + w_2 x_2 + b = 0$$

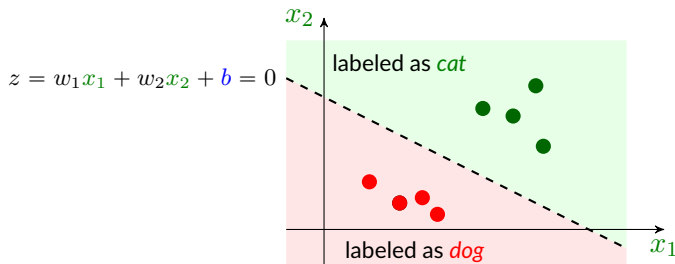
for some $w_1, w_2 > 0$ and $b < 0$



Perceptron draws a **line**, and then classifies every point above it with **label 1** and every point below it with **label 0**

Perceptron: Geometrical Interpretation

So, if we have a perceptron with *learnable parameters* $w_1, w_2 > 0$, and $b < 0$



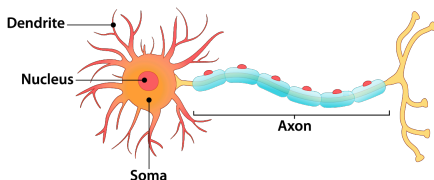
For instance in the example of image classification

- Every • represents a *data-point* $\equiv$ pixel vector: we should think of a two-pixel image 😊
- Perceptron gets the values of the two pixels and predicts whether it is an image of a *dog* or a *cat*

In realistic problems, this viewpoint extends to large-dimensional space

Perceptron: A Bit of History

- Perceptron machine was implemented by **Frank Rosenblatt** in 1957
- He wrote a paper in 1958 illustrating the machine and its algorithm
- The original idea is however older than that
 - ↳ It was proposed in 1943 by **Warren McCulloch** and **Walter Pitts**
 - ↳ They introduced it to abstractly describe biological neurons
 - ↳ This was why perceptron is also called **McCulloch-Pitts neuron**



- Perceptron was a breakthrough in the long-time ongoing attempt to understand the functionality of brain; however,
*for us, **perceptron** is simply a **linear classification model***

Components of Binary Classification

Back to binary classification: we have the first two components

- 1 A dataset with N -dimensional inputs $\mathbf{x}_i$ and binary labels $y_i \in \{0, 1\}$

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

- 2 Perceptron as the model

$$y = s(\mathbf{w}^T \mathbf{x} + b)$$

We are now looking for the third component, i.e., the loss function

Binary Classification: Loss

In general, we can use any **loss function**

- *squared error*
- *Kullback-Leibler divergence* $\leftarrow$ we are going to learn it soon
- *error indicator*
- ...

Let's use the one initially used in Rosenblatt's machine, i.e., **error indicator**¹

For two binary variables y and $\hat{y}$, the loss function is

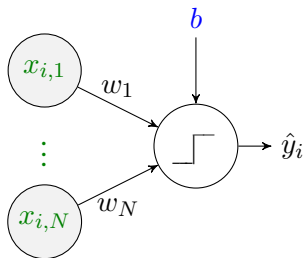
$$\mathcal{L}(\hat{y}, y) = \mathbb{1}(\hat{y} \neq y) = \begin{cases} 1 & \hat{y} \neq y \\ 0 & \hat{y} = y \end{cases}$$

¹We will realize later that this is not a good choice

Binary Classification: Loss

How does this **loss** look like when we classify via perceptron?

Let $(\mathbf{x}_i, y_i)$ be a data-point in **dataset**: we give $\mathbf{x}_i$ to the perceptron



and get $\hat{y}_i$. The loss says whether we got the **same label (0)** or **not (1)**

$$\mathcal{L}(\hat{y}_i, y_i) = \mathbb{1}(\hat{y}_i \neq y_i) = \begin{cases} 1 & \hat{y}_i \neq y_i \\ 0 & \hat{y}_i = y_i \end{cases}$$

Training a Model

Up to now, we have learned the main components of ML problems and seen a classical example, i.e., classification

*We now want to understand how we can **tune** the **learnable parameters** of a model for our problem.*

As mentioned, **tuning** the **learnable parameters** of a model is called
training of the model

*Before starting with **training**, let's review some basics of probability theory*

Recap: Probability Theory

Discrete Random Variables

Discrete random variable x is described by *probability mass function* $P(x)$

$$\Pr\{x = x_0\} = P(x_0)$$

Assuming that $x \in \mathbb{X}$, we have

$$\sum_{x \in \mathbb{X}} P(x) = 1$$

Continuous Random Variables

Continuous random variable x is described by *probability density function* $P(x)$

$$\Pr\{a < x \leq b\} = \int_a^b P(x) dx$$

We have in this case

$$\int_{-\infty}^{+\infty} P(x) dx = 1$$

When we talk about a *general* random variable, we call $P(x)$ the *distribution*

Recap: Probability Theory

A random vector $\mathbf{x}$ is a vector of random variables and its distribution

$$P(\mathbf{x}) = P(x_1, \dots, x_N)$$

is the *joint distribution of the entries*

Assume $\mathbf{x} \in \mathbb{R}^N$ is a vector of random variables, its *expectation* is

$$\boxed{\text{Discrete}} \quad \mathbb{E}\{\mathbf{x}\} = \sum_{\mathbf{x} \in \mathbb{X}^N} \mathbf{x} P(\mathbf{x}) \quad \boxed{\text{Continuous}} \quad \mathbb{E}\{\mathbf{x}\} = \int \mathbf{x} P(\mathbf{x}) d\mathbf{x}$$

and we can extend the definition to any function of $\mathbf{x}$, i.e.,

$$\mathbb{E}\{f(\mathbf{x})\} = \int f(\mathbf{x}) P(\mathbf{x}) d\mathbf{x}$$

Recap: Law of Large Numbers

Consider a random sequence $x_1, \dots, x_I$

we call it **independent** and **identically distributed** (i.i.d.) with $x \sim P(x)$

if x_i 's are generated **independently** all with the **same distribution** $P(x)$

Law of Large Numbers

Assume $x_1, \dots, x_I$ is an i.i.d. sequence with $x \sim P(x)$; then, we have²

$$\frac{1}{I} \sum x_i \rightarrow \mathbb{E}\{x\}$$

as $I \rightarrow \infty$

In simple words: if we **arithmetically average** too many samples of a random process, we get a value very close to its **expectation**

²Of course under some conditions which we assume holding

Training a Model

Let us now use the probability theory to derive a meaningful approach of *model training*: for sake of simplicity, let's assume that we have *dataset*

$$\mathbb{D} = \{(x_i, y_i) : i = 1, \dots, I\}$$

with *scalar (one-dimensional) inputs*, i.e., x_i 's are real numbers

Let us further denote our model as

$$y = f(x|\mathbf{w})$$

where in this notation

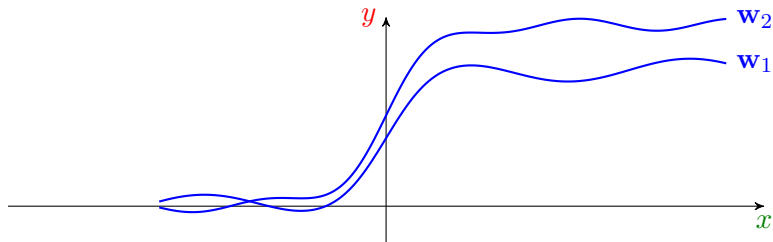
- $\mathbf{w}$ contains all *learnable* parameters that are to be learned in *training*
- we drop the *hyperparameters*, since *they are yet fixed by genie*

Training a Model

The *one-dimensional* assumption helps us visualize the *model*

$$y = f(x|\mathbf{w})$$

With scalar *input*, we can visualize the model as



As *learnable* parameters change, the model represents different functions

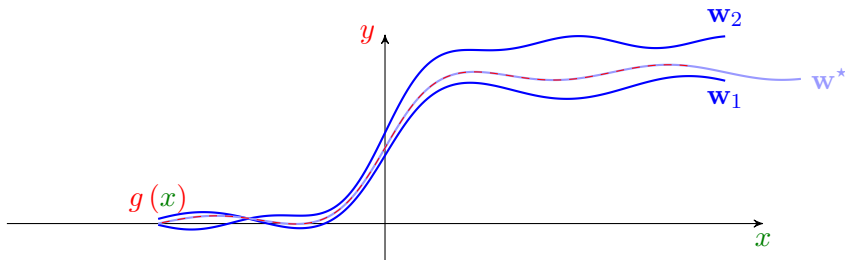
Training a Model

In reality, the **labels** and **inputs** are related through a function $g(\cdot)$

$y = g(x) \rightsquigarrow$ function $g(\cdot)$ is however **unknown** to us

In fact, **our whole learning task** is to **learn** it from the **dataset**!

- + What if we knew **function** $g(\cdot)$? How would have we trained our model?
- Well! We would have tuned **w** until the model matches $g(\cdot)$



Training a Model

We could represent such $\mathbf{w}^*$ using the *loss*

Point-wise Loss

Let $\mathcal{L}$ be loss function: the model with *learnable parameter* $\mathbf{w}$ gives us the label $\hat{y}_0 = f(x_0|\mathbf{w})$ for *input* x_0

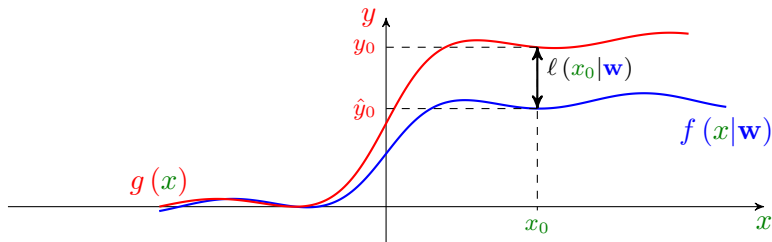
- It may be different from the *true label* $y_0 = g(x_0)$
- The *loss* between the two is given by

$$\ell(x_0|\mathbf{w}) = \mathcal{L}(\hat{y}_0, y_0) = \mathcal{L}(f(x_0|\mathbf{w}), g(x_0))$$

Training a Model: *Point-wise Loss*

$$\ell(x_0|\mathbf{w}) = \mathcal{L}(\hat{y}_0, y_0) = \mathcal{L}(f(x_0|\mathbf{w}), g(x_0))$$

Let's visualize this loss



Training a Model: Risk

We would like our *model* to recover the *true label as closely as possible*: so, the best option at point x_0 is to find the choice of $\mathbf{w}$ that minimizes $\ell(x_0|\mathbf{w})$

- + But x_0 is a *single* point! What about other *inputs*?!
 - Right! We should *learn* $\mathbf{w}$ for any x_0
- + How can we do it?
 - We treat x_0 as a random variable with some distribution $P(x_0)$, and minimize the *expected loss* often called *risk*

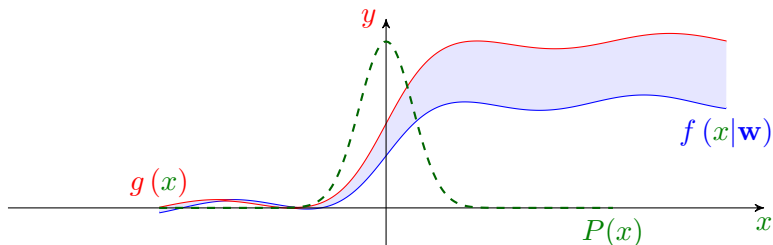
Risk

For given learnable parameter $\mathbf{w}$, the *risk* is defined as

$$R(\mathbf{w}) = \mathbb{E} \{ \ell(x_0|\mathbf{w}) \} = \int \ell(x_0|\mathbf{w}) P(x_0) dx_0$$

Training a Model: Risk Minimization

Let's visualize the **risk**



Risk Minimization

Ideally, the training is formulated as the minimization of **risk**

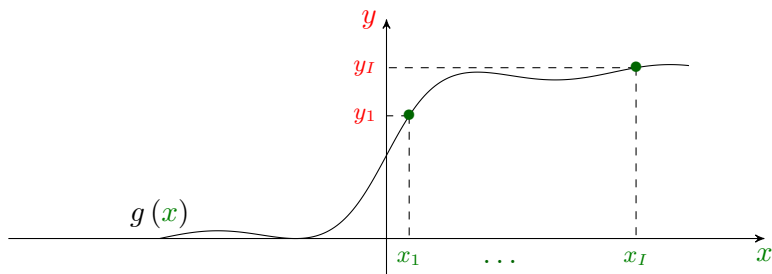
$$\mathbf{w}^{\star} = \operatorname{argmin}_{\mathbf{w}} R(\mathbf{w})$$

Training a Model: *Empirical Risk*

- + Bravo! But, the *training* seems *infeasible*, since we have neither the *true function* $g(\cdot)$, nor the *distribution* $P(x)$!
- Right! But, we could handle this approximately using the *LLN*

Let's look at what we have: the *dataset* which contains *samples* of $g(\cdot)$

$$\mathbb{D} = \{(x_i, y_i) : i = 1, \dots, I\}$$



Training a Model: *Empirical Risk*

We can determine *point-wise losses* for points in the *dataset*

$$\ell(x_i|\mathbf{w}) = \mathcal{L}(f(x_i|\mathbf{w}), g(x_i)) = \mathcal{L}(f(x_i|\mathbf{w}), y_i)$$

Now, assume that data-points are drawn from i.i.d. *unknown* $P(x)$

LLN suggests that *average* of *point-wise losses* converges to the *risk* when we have a large enough number of data-points, i.e.

$$\frac{1}{I} \sum_{i=1}^I \ell(x_i|\mathbf{w}) \rightarrow \mathbb{E} \{ \ell(x|\mathbf{w}) \} = R(\mathbf{w})$$

We call this *arithmetic average* the *empirical risk*

Empirical risk is the best *estimate* of *risk* that we get from our *dataset*

Training a Model: *Empirical Risk Minimization*

Empirical Risk

Let $\mathbf{w}$ includes all learnable parameters of the model, and the *dataset* be

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

for *loss function* $\mathcal{L}$, the *empirical risk* is defined as

$$\hat{R}(\mathbf{w}) = \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f(\mathbf{x}_i | \mathbf{w}), y_i)$$

Training a Model: *Empirical Risk Minimization*

The *training* is performed by minimizing the empirical risk

Empirical Risk Minimization

To *train* a model on the *dataset*, we *minimize* the *empirical risk* computed from the *dataset*

$$\begin{aligned}\mathbf{w}^{\star} &= \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) && \text{(Training)} \\ &= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f(\mathbf{x}_i | \mathbf{w}), y_i)\end{aligned}$$

Wrap-Up: *Universal Scheme for ML*

So, we know pretty much all the theory for *supervised* learning:

- ① We build the main three components:
 - ① Dataset
 - ② Model
 - ③ Loss Function
- ② We determine the *empirical risk* of the *model* using *loss function*
- ③ We *train* the *model* by minimizing the *empirical risk*

We call this universal approach ML 1-2-3!

The theory is *pretty short*; however,

how to *execute* each step of ML 1-2-3 is a *pretty long* story

that we are going to learn in the remaining of this course!